

Deploying a Go “Hasher” Chaincode on Hyperledger Fabric

Version: 1.0 **Date:** October 26, 2023

1. Introduction

This document provides a step-by-step guide to creating, deploying, and testing a simple Hyperledger Fabric chaincode written in Go. The chaincode, named hasher, accepts a string input and returns its SHA256 hash. We will use the standard Fabric test-network for deployment and testing via the command line (CLI) and discuss how to interact via HTTP conceptually.

Goal: Deploy hasher chaincode with function HashString(input string) returning the SHA256 hash.

2. Prerequisites

Before starting, ensure you have the following installed and configured:

- **Docker & Docker Compose:** For running Fabric network components. (<https://docs.docker.com/get-docker/>)
 - Verify: `docker --version, docker compose version`
- **Go Programming Language:** Version 1.17+ (1.19+ recommended). (<https://go.dev/dl/>)
 - Verify: `go version`
- **Git:** For cloning repositories. (<https://git-scm.com/downloads>)
 - Verify: `git --version`
- **Hyperledger Fabric Samples & Binaries:** Contains the test network and required tools (peer, orderer, etc.).

```
# Create a directory (optional)
mkdir -p $HOME/hyperledger
cd $HOME/hyperledger

# Download install script and run (e.g., Fabric v2.4.7, CA
v1.5.5)
# Check Fabric docs for compatible versions!
curl -sSL
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-
fabric.sh | bash -s -- -f 2.4.7 -c 1.5.5

# Add Fabric binaries to your PATH
```

```

export PATH=$PWD/bin:$PATH
echo 'export PATH=$HOME/hyperledger/bin:$PATH' >> ~/.profile # Or
your shell profile
source ~/.profile # Or your shell profile

```

3. Step 1: Create the Go Chaincode (hasher.go)

1. **Create Directory Structure:**

```

bash      cd $HOME/hyperledger/fabric-
samples  mkdir -p chaincode-go/hasher      cd chaincode-
go/hasher

```
2. **Initialize Go Module:**

```

bash      # Replace 'example.com/hasher' if
desired  go mod init example.com/hasher    go get
github.com/hyperledger/fabric-contract-api-go@latest    go mod
tidy

```

3. **Create hasher.go:**

```

package main

import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"

    "github.com/hyperledger/fabric-contract-api-go/contractapi"
)

// SmartContract provides functions for hashing
type SmartContract struct {
    contractapi.Contract
}

// HashString calculates SHA256 hash of inputString
func (s *SmartContract) HashString(ctx
contractapi.TransactionContextInterface, inputString string)
(string, error) {
    if inputString == "" {
        return "", fmt.Errorf("input string cannot be empty")
    }

    hasher := sha256.New()
    hasher.Write([]byte(inputString))
    hashBytes := hasher.Sum(nil)
    hashString := hex.EncodeToString(hashBytes)

    fmt.Printf("HashString called for: %s, Result: %s\n",
inputString, hashString)
    return hashString, nil
}

func main() {
    chaincode, err := contractapi.NewChaincode(&SmartContract{})
    if err != nil {
        fmt.Printf("Error creating hasher chaincode: %s",
err.Error())
        return
    }
}

```

```

        if err := chaincode.Start(); err != nil {
            fmt.Printf("Error starting hasher chaincode: %s",
err.Error())
        }
    }
}

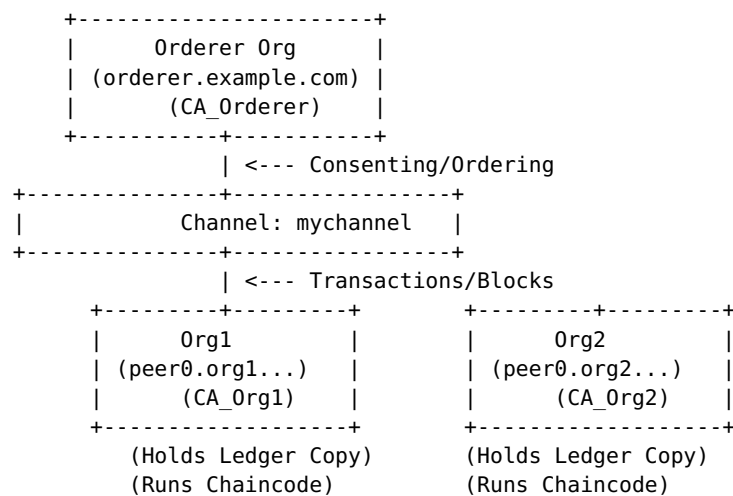
```

4. Step 2: Setup the Fabric Test Network

We use the test-network from fabric-samples.

1. **Navigate:** `bash cd $HOME/hyperledger/fabric-samples/test-network`
2. **Cleanup (Optional):** `bash ./network.sh down`
3. **Start Network & Create Channel:** `bash ./network.sh up createChannel -c mychannel -ca` This command starts Org1 Peer, Org2 Peer, an Orderer, CAs for each, and creates the mychannel channel, joining both peers to it.
4. **Verify Containers:** `bash docker ps` (You should see containers for peers, orderer, CAs).

[Diagram: Fabric Test Network Overview]



5. Step 3: Deploy the Chaincode (Lifecycle Process)

This involves packaging, installing, approving, and committing the chaincode definition.

1. **Set Environment Variables:** Ensure Fabric tools are in PATH and set the config path. `bash # If not already set from prerequisites:`
`# export PATH=$HOME/hyperledger/bin:$PATH # Run from test-`

```
network directory      cd $HOME/hyperledger/fabric-samples/test-  
network      export FABRIC_CFG_PATH=$PWD/../config/
```

2. **Package Chaincode:**

```
bash      peer lifecycle chaincode package  
hasher.tar.gz \      --path ../chaincode-go/hasher/ \      --  
lang go lang \      --label hasher_1.0 (Creates hasher.tar.gz)
```

3. Install on Peers:

◦ Org1 Peer0:

```
# Set Org1 Admin context  
export CORE_PEER_TLS_ENABLED=true  
export CORE_PEER_LOCALMSPID="Org1MSP"  
export  
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganiza  
tions/org1.example.com/peers/peer0.org1.example.com/tls/ca.cr  
t  
export  
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganization  
s/org1.example.com/users/Admin@org1.example.com/msp  
export CORE_PEER_ADDRESS=localhost:7051  
  
# Install  
peer lifecycle chaincode install hasher.tar.gz  
  
# ---> IMPORTANT: Note the Package ID outputted here! <---  
# Example: hasher_1.0:aabbccddeeff...  
  
# Store Package ID in variable (adjust grep/sed if needed)  
export CC_PACKAGE_ID=$(peer lifecycle chaincode  
queryinstalled | grep 'Package ID: hasher_1.0' | sed -n  
's/Package ID: \(.*\) , Label: .*/\1/p')  
echo "Package ID: ${CC_PACKAGE_ID}" # Verify it captured  
correctly
```

◦ Org2 Peer0:

```
# Set Org2 Admin context  
export CORE_PEER_LOCALMSPID="Org2MSP"  
export  
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganiza  
tions/org2.example.com/peers/peer0.org2.example.com/tls/ca.cr  
t  
export  
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganization  
s/org2.example.com/users/Admin@org2.example.com/msp  
export CORE_PEER_ADDRESS=localhost:9051  
  
# Install  
peer lifecycle chaincode install hasher.tar.gz
```

4. Approve for Organizations:

◦ Org1 Approve:

```
# Set Org1 Admin context (if needed)  
export CORE_PEER_LOCALMSPID="Org1MSP"  
export  
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganiza
```

```
tions/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

# Approve
peer lifecycle chaincode approveformyorg -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com --tls \
  --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
  --channelID mychannel --name hasher --version 1.0 \
  --package-id ${CC_PACKAGE_ID} \
  --sequence 1
```

o **Org2 Approve:**

```
# Set Org2 Admin context
export CORE_PEER_LOCALMSPID="Org2MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp
export CORE_PEER_ADDRESS=localhost:9051

# Approve
peer lifecycle chaincode approveformyorg -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com --tls \
  --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
  --channelID mychannel --name hasher --version 1.0 \
  --package-id ${CC_PACKAGE_ID} \
  --sequence 1
```

5. **Check Commit Readiness:** bash # Use either Org1 or Org2 context peer lifecycle chaincode checkcommitreadiness --channelID mychannel \ --name hasher --version 1.0 --sequence 1 --tls \ --cafile "\${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" --output json (Verify approvals are true for Org1MSP and Org2MSP).

6. **Commit Chaincode Definition:**

```
# Set Org1 Admin context (for submitting peer)
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/or
```

```

g1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

# Commit (requires targeting peers from approving orgs)
peer lifecycle chaincode commit -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/o
rderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
--channelID mychannel --name hasher --version 1.0 --sequence 1
\
--peerAddresses localhost:7051 --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org1.example.com/peers/pe
er0.org1.example.com/tls/ca.crt" \
--peerAddresses localhost:9051 --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/peers/pe
er0.org2.example.com/tls/ca.crt"

```

7. **Query Committed:** bash # Use either Org1 or Org2 context
 peer lifecycle chaincode querycommitted --channelID mychannel --
 name hasher \ --cafile
 "\${PWD}/organizations/ordererOrganizations/example.com/orderers/o
 rderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem"
 (Verify the definition for hasher, sequence 1, is listed).

[Diagram: Chaincode Lifecycle Flow]



6. Step 4: Test Chaincode using CLI

Invoke the HashString function using the peer command.

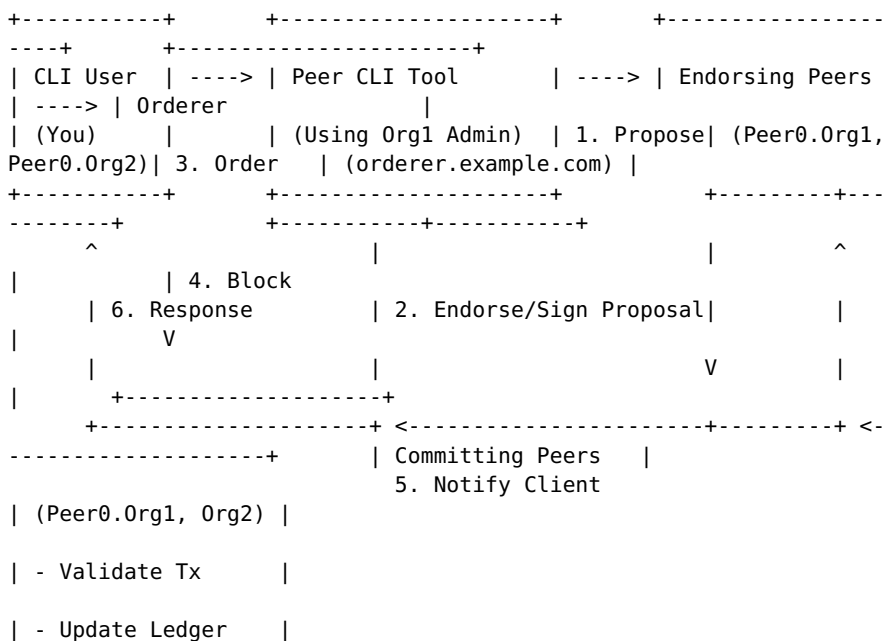
1. **Set Peer Context (e.g., Org1):**

```
bash export
CORE_PEER_TLS_ENABLED=true export
CORE_PEER_LOCALMSPID="Org1MSP" export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp export
CORE_PEER_ADDRESS=localhost:7051
```
2. **Invoke HashString:**

```
bash peer chaincode invoke -o
localhost:7050 \ --ordererTLSHostnameOverride
orderer.example.com --tls \ --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
--channelID mychannel --name hasher \ --peerAddresses
localhost:7051 --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \ --peerAddresses
localhost:9051 --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \ -c
'{"function": "HashString", "Args": ["hello world"]}'
```
3. **Check Output:** Look for status:200 and the hash in the payload. #

```
Expected Output (Hash for "hello world"): ... INFO 001
Chaincode invoke successful. result: status:200
payload: "b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9"
```

[Diagram: CLI Invocation Flow]



+-----+

7. Step 5: Test using an HTTP Request (Conceptual)

Direct HTTP access to chaincode is not standard. You need an intermediary application (API server) built with a Fabric SDK (Go, Node.js, Java, Python).

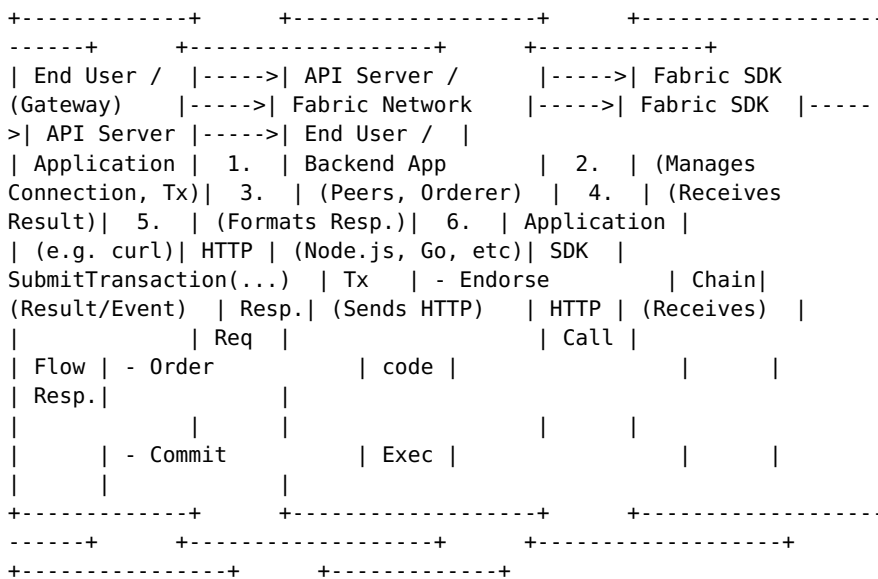
1. SDK Application Role:

- Connects to the Fabric network (using connection profile).
- Manages user identities (wallet).
- Provides an HTTP API endpoint (e.g., POST /hash).
- Uses the SDK's `contract.SubmitTransaction()` method to call the chaincode.
- Handles the transaction lifecycle (endorsement, ordering, commit notification).
- Returns the chaincode result in the HTTP response.

2. **Example curl to a Hypothetical API Server:** (Assuming an API server runs on `http://localhost:8080` with a `/hash` endpoint) `bash curl -X POST \ http://localhost:8080/hash \ -H 'Content-Type: application/json' \ -d '{ "inputString": "test via http" }'`

3. **Expected HTTP Response (JSON):** `json { "hash": "146a19586a7a6877f7b861e3aa16c757d540d6c191a66816b6f8d38d3f266f18" }`

[Diagram: Conceptual HTTP Request Flow]



(Note: Building the SDK application requires separate development effort using Fabric SDK libraries).

8. Cleanup

When finished, stop and remove the test network components:

```
cd $HOME/hyperledger/fabric-samples/test-network  
./network.sh down
```

This removes containers, crypto material, and channel artifacts generated by the script. Your chaincode source code remains intact.